

CSCE 608 DATABASE SYSTEMS · MODULE 01 · LECTURE NOTES

What a Database Is For

The problems that justify a hundred thousand lines of code.

MODULE OUTCOMES

State the problems a DBMS solves that a filesystem does not.

Explain data independence and why it matters.

Describe the layered architecture of a DBMS.

Explain the relational model's actual contribution.

Situate the course's modules within the architecture.

1 Why not just use files?

The honest way to motivate a database is to start with files and watch the requirements accumulate until you have rebuilt one badly.

Problem	What goes wrong
No structure	Every program that touches the file must agree on its format, and that agreement lives nowhere except in the programs. Changing the format means finding and changing all of them.
No concurrent access	Two processes writing simultaneously interleave their writes and corrupt the file. A reader sees a record that is half old and half new. Neither is detectable after the fact.
No atomicity	An update that touches three records can be interrupted after two. The file is now in a state no program was written to expect, and nothing records which operation was in flight.
No query capability	Every new question requires new code, and every access path is hand-written. Finding records by a field nobody anticipated means a full scan or a new hand-built index.
No scale beyond memory	Sorting or joining data larger than RAM is a substantial project in its own right (Module 06).

THE ORGANISING QUESTION OF THIS COURSE**How do you make a machine that loses power behave as though it did not?**

Nearly every structural decision in a database descends from that question, and the chain is worth holding in view:

Durability requires a log, because you cannot update data in place and survive a crash halfway (Module 11). **The log's cost requires a buffer pool**, because writing every change to disk immediately is unaffordable (Module 02). **Concurrent access to the buffer pool requires latching** (Module 04). **The cost of locking requires multi-version concurrency control** (Module 10).

Each module adds one link. The architecture stops looking like an arbitrary pile of components once you see it as a sequence of forced moves from a single requirement.

2 Data independence

CODD'S CONTRIBUTION WAS INDEPENDENCE, NOT TABLES

The relational model is usually described as 'data in tables', which misses what was actually new. Tables were not the point.

Before 1970, programs navigated data structures explicitly: follow this pointer, traverse this chain, read the next record in this linked list. The access path was written into the application, in the application's own code.

So changing the physical organisation meant rewriting every program that touched the data. Adding an index, reordering records, or splitting a file was a project across the whole codebase.

Codd separated what you want from how to obtain it. A query states which rows satisfy a condition, not which pointers to follow. The system chooses the access path, at query time, from whatever structures currently exist.

This is the entire reason query optimisation exists (Module 08). It is also why a database administrator can add an index and make every existing application faster without any of them being modified or even recompiled — which is a remarkable property and easy to take for granted.

Level	Describes	Independence it provides
Physical schema	Files, pages, record layout, indexes, compression.	—
Logical schema	Tables, columns, types, constraints, relationships.	Physical data independence: the storage layout can be reorganised entirely without changing any query. This is the one that was genuinely achieved , and it is the foundation of the whole field.

Level	Describes	Independence it provides
External schema (views)	What a particular application or user is shown.	Logical data independence: the logical schema can change without breaking applications. Achieved only partially — views can hide added columns and renamings, and cannot hide a genuine restructuring.

3 The architecture

Layer	Responsibility	Covered in
Parser and binder	SQL text to a validated logical plan: syntax, name resolution, type checking.	Module 05
Optimiser	Logical plan to physical plan , choosing join order, join algorithms, and access paths by estimated cost.	Module 08
Executor	Runs the physical plan: operators producing and consuming tuples.	Modules 06, 07
Access methods	B+trees, hash indexes, sequential scans — the ways data can be reached.	Modules 03, 04
Buffer pool	Which pages are in memory and which are evicted. The boundary between the database and the disk.	Module 02
Transaction manager	Concurrency control and isolation.	Modules 09, 10
Recovery manager	Logging, checkpointing, and surviving a crash.	Module 11
Disk manager	Pages, files, and the storage hardware.	Module 02

READ THE ARCHITECTURE PAPER IN WEEK ONE

Hellerstein, Stonebraker and Hamilton's *Architecture of a Database System* is about a hundred pages, free, and the best overview of how these components relate.

Read it before you understand any of the pieces. It is a map, and a map is most useful before you are lost.

Then read it again at the end of the course. It will be a noticeably different document, because every sentence that was a bare assertion the first time will have an implementation behind it.

This is unusually specific advice for a survey paper, which is why it is in the required materials rather than buried in a module's resource list.

4 The design space

Axis	The choice	Trade
Workload	OLTP (many small transactions) versus OLAP (few large analytical queries).	Determines essentially every other choice on this list. Postgres versus Snowflake.
Layout	Row store versus column store.	Rows for reading whole records; columns for scanning one attribute across millions of records (Module 12). MySQL versus DuckDB.

Axis	The choice	Trade
Storage	Disk-oriented versus in-memory.	In-memory systems eliminate the buffer pool and change the cost model entirely. Postgres versus Redis or VoltDB.
Index	B+tree versus LSM tree.	B+trees favour reads, LSM trees favour writes (Module 04). Postgres versus RocksDB.
Concurrency	Locking versus multi-version.	Locking blocks readers against writers; MVCC does not, at the cost of storing old versions (Modules 09, 10).
Scale	Single node versus distributed.	Distribution buys capacity and costs correctness guarantees, latency, and operational complexity (Module 13).

KNOW WHEN NOT TO USE A DATABASE

A plain file is entirely appropriate for configuration, for append-only logs, and for anything that one process writes and nobody queries. Reaching for a database there adds a dependency and solves nothing.

SQLite is appropriate for an enormous range of applications. It is an embedded library rather than a server, it implements most of this course in about 150,000 lines, and it is by a very wide margin the most widely deployed database in the world — it is in every phone, every browser, and most applications.

A distributed database is rarely necessary. A single machine with a modern NVMe drive and a few hundred gigabytes of RAM handles workloads that genuinely required a cluster fifteen years ago, and it does so without giving up transactions, joins, or straightforward debugging.

The reflex to distribute is usually wrong, and it is expensive in ways that are not obvious until you are committed. Module 13 makes the case precisely, with the trade-offs stated rather than gestured at.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lecture 1, Course Introduction and Relational Model	Pavlo's opening lecture covers §1 and §2 with more history and more jokes. Watch it before reading these notes.
Hellerstein, Stonebraker & Hamilton — Architecture of a Database System (free)	The map of §3. Read it this week.
Codd — A Relational Model of Data for Large Shared Data Banks (1970, free)	The original paper. Short, and the data independence argument of §2 is clearer in the original than in most retellings.
The Red Book — editorial introduction	Stonebraker and Hellerstein's commentary on what has actually mattered in fifty years of the field. Opinionated and worth reading.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a program that stores records in a flat file and supports lookup by key. Then add concurrent writers and demonstrate corruption.
- 2 Add a crash point to the same program — kill it mid-update — and show that the file is left in an inconsistent state.
- 3 Install SQLite and PostgreSQL. Create the same schema in both and compare what EXPLAIN tells you about the same query.
- 4 Take a query against a table with no index, measure it, add an index, and measure again. Confirm that the query text did not change — this is physical data independence, demonstrated.
- 5 Read the architecture paper and draw the layer diagram of §3 from memory. Identify which layer each of this course's thirteen modules belongs to.
- 6 For three systems you have used or heard of, place each on all six axes of §4 and justify each placement.
- 7 Find a project you know that uses a distributed database. Estimate whether a single machine would have sufficed, and state the numbers you based it on.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give five things a filesystem does not provide that a DBMS does.
- 2 State the organising question of this course and the chain of consequences from it.
- 3 What was Codd's actual contribution, and why is 'data in tables' a poor summary?
- 4 Define physical and logical data independence, and say which was genuinely achieved.
- 5 Why does query optimisation exist at all?
- 6 Name the eight layers of a DBMS from the top down.
- 7 Give six axes on which database systems differ, with an example of each.
- 8 Give three situations where a full DBMS is the wrong choice.